

Advanced Database Models, Systems, and Applications

UNIT 5

Active Database Concepts and Triggers

Active

Database

- ✓ Active database system is a database system that supports reactive behavior through ECA-rules.
- ✓ An active database is a database that includes an event-driven architecture which can respond to conditions both inside and outside the database.
- ✓ Possible uses include security monitoring, alerting, statistics gathering and authorization.
- ✓ Active Database is a database consisting of set of triggers.

ECA Rules

■ Event occurs in databases

- addition of new row, deletion of row by DBMS

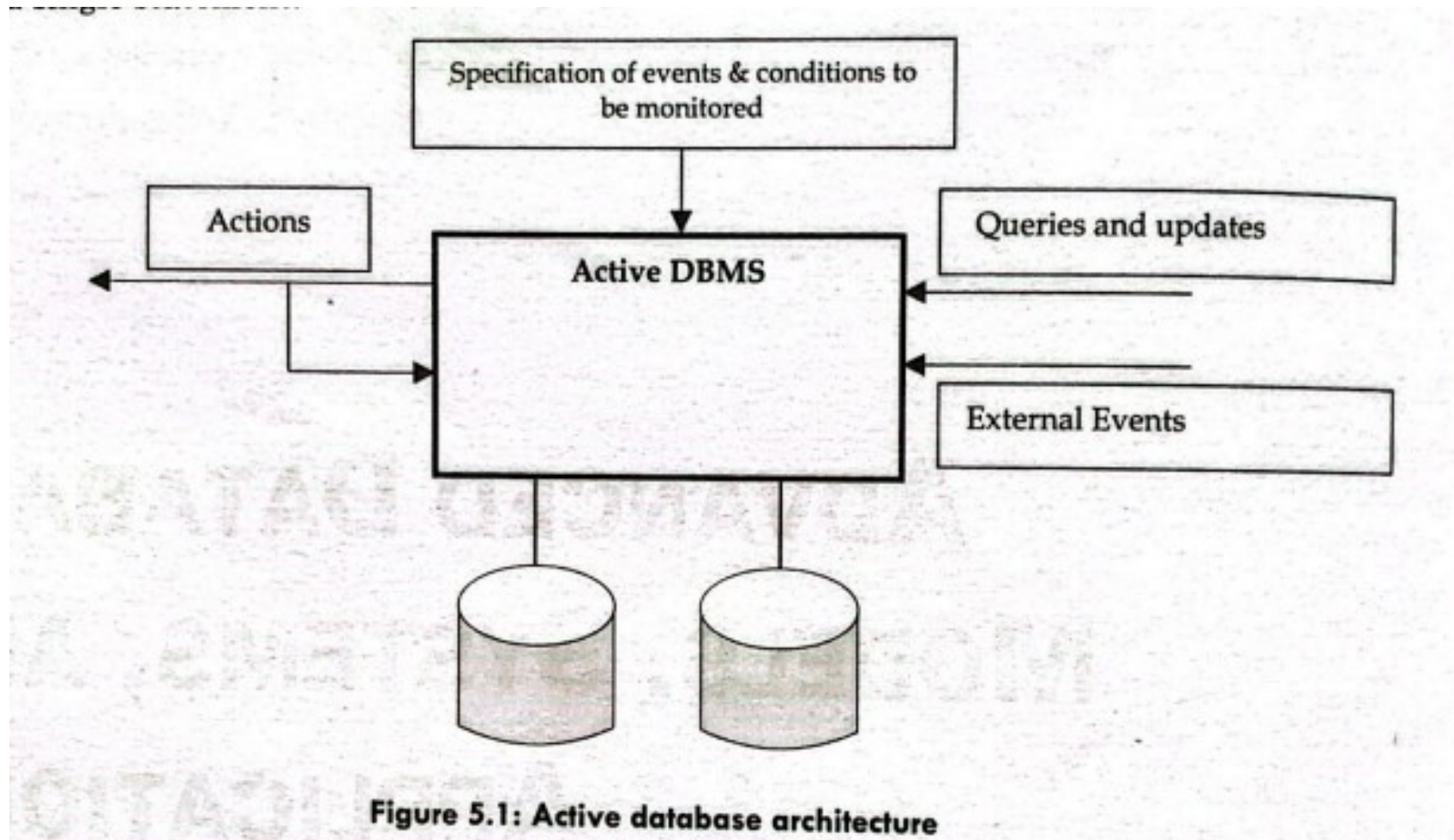
■ Conditions are checked

- SQL condition

■ Actions are executed if conditions are satisfied

- SQL + procedures
- All data actions performed by the trigger execute within the same transaction in which the trigger fires,

Active Database



Trigge

- ✓ A trigger defines an action the database should take when some database-related event (such as inserts, updates, deletes) occurs.
- ✓ A block of code that is attached to an event. When that event occurs the trigger code is fired.

Triggers follow an Event-condition-action (ECA) model

- **Event:**
 - Database modification
 - E.g., insert, delete, update),
- **Condition:**
 - Any true/false expression
 - Optional: If no condition is specified then condition is always true
- **Action:**
 - Sequence of SQL statements that will be automatically executed

Use of

1. Trigger

- Write information about (sensitive) data modifications to an audit table
- May include old and new values, user, timestamp
- E.g. new and old salary

2. Data Integrity

- Implement checks on data against business rules
- Can compare with live database values
- NEW and OLD values can be compared
- E.g. prices must not go down

3. Referential integrity

- Allows implementation of a "cascade update"
- E.g. if author ID (aID) is changed, appropriately change authID in foreign key

4. Derived data

Use of Trigger

5. Security

- Logging of database access
- E.g. date and time each user logs on
- E.g. deny access at weekend

6. Maintaining synchronous replicates

- In a distributed database

7. Generating statistics on table access

Types of

• Triggers can be Trigger

- **Row-level**

- FOR EACH ROW specifies a row-level trigger

- **Statement-level**

- Default (when FOR EACH ROW is not specified).
- A statement-level trigger is **fired whenever a trigger event occurs on a table regardless of how many rows are affected**. In other words, a statement-level trigger executes once for each transaction.
- For example, if you update 1000 rows in a table, then a statement-level trigger on that table would only be executed once

- Row level triggers

- Executed separately for each affected row

- Statement-level triggers

- Execute once for the SQL statement,

Example of Row Level

Trigger

- ✓ create table Student (id number, name varchar2(100));
- ✓ insert into Student values (1,'Ram');
- ✓ insert into Student values (2,'Sita');
- ✓ create table Log(id number, old_value varchar2(100), new_value varchar2(100), updateDate TimeStamp);
- ✓ create user c###ouser identified by adbms;
- ✓ grant unlimited tablespace to c###ouser;
- ✓ grant resource, connect, dba to c###ouser;

Create a trigger

create or replace trigger Tg_Student before update
on Student for each row

begin

 insert into Log values (:old.id,
 :old.name, :new.name, CURRENT_TIMESTAMP);

end;

update student set name='Hari'; [See the log table }

Example Statement level trigger

Suppose, you want to restrict users to update credit of customers from 28th to 31st of every month so that you can close the financial month.

To enforce this rule, you can use this statement-level trigger:

```
CREATE OR REPLACE TRIGGER customers_credit_trg
  BEFORE UPDATE OF credit_limit
  ON customers
DECLARE
  l_day_of_month NUMBER;
BEGIN
  -- determine the transaction type
  l_day_of_month := EXTRACT(DAY FROM sysdate);

  IF l_day_of_month BETWEEN 28 AND 31 THEN
    raise_application_error(-20100,'Cannot update customer credit from 28th to 31st');
  END IF;
END;
```

Note that Oracle automatically rollbacks the update because we call the `raise_application_error` procedure inside the trigger.

Temporal Database Concepts

Temporal

Database

- ✓ A temporal database is a database with built-in support for handling time sensitive data.
- ✓ Temporal databases, encompass all DB applications that require some aspect of time when organizing their information.
- ✓ Temporal DB applications have been developed since the early days of database usage. However, in creating these applications, it was mainly left to the application developers to discover, design, program, and implement the temporal concepts.
- ✓ Temporal databases can be used for a variety of applications, such as auditing, compliance, historical analysis, fraud detection, and decision making.
- ✓ Temporal databases can be more complex to manage than traditional databases. This is because they need to store additional information about time, and they need to be able to track changes to data over time.

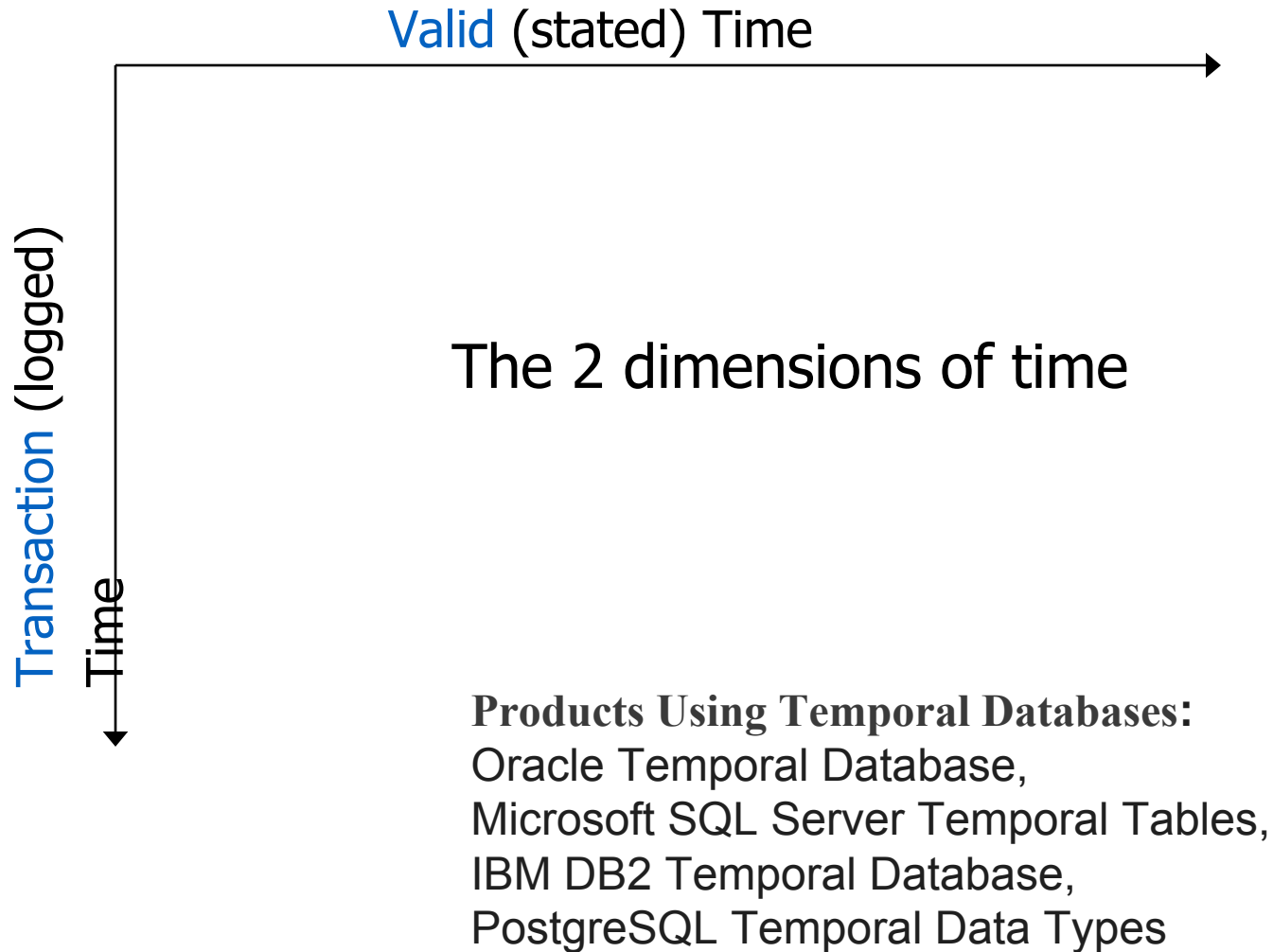
Non Temporal Database

- Store only a single state of the real world, usually the most recent state
- Classified as snapshot databases
- No old values
- Application developers and database designers need to code for time varying data requirements eg history tables, forecast reports etc

Temporal Database Concept

- stores upto two dimensions of time i.e VALID (stated) time and TRANSACTION (logged) time
- Temporal databases have two main types of time: valid time and transaction time.
- Valid time denotes the time period during which a fact is true with respect to the real world.
- Transaction time is the time period during which a fact is stored in the databases.
- Classified as historical, rollback or bi-temporal
- No need for application developers or database designers to code for time varying

What are temporal databases?



Time Representation..

The temporal aspects usually include valid time and transaction time.

These attributes can be combined to form bi-temporal data.

- **Valid time** is the time period during which a fact is true with respect to the real world.
- **Transaction time** is the time period during which a fact stored in the database is considered to be true.
- **Bi-temporal data** combines both Valid and Transaction Time

Exempl

Now let's see an example of a person, John:

- John was born on April 3, 1992 in Chennai.
- His father registered his birth after three days on April 6, 1992.
- John did his entire schooling and college in Chennai.
- He got a job in Mumbai and shifted to Mumbai on June 21, 2015.
- He registered his change of address only on Jan 10, 2016.

Source:

<https://www.mytecbits.com/oracle/oracle-database/what-is-temporal-database>

Name	City	Valid From	Valid Till
John	Chennai	April 3, 1992	June 20, 2015
John	Mumbai	June 21, 2015	∞

Valid Time and Transaction Time

Johns father registers his birth on 6th April 1992:

Person(John, Chennai, 3-Apr-1992, ∞ , 6-Apr-1992, ∞).

On January 10, 2016 John reports his new address in Mumbai:

Person(John, Mumbai, 21-June-2015, ∞ , 10-Jan-2016, ∞).

The original entry is updated.

Person(John, Chennai, 3-Apr-1992, 20-June-2015, 6-Apr-1992, 10-Jan-2016).

Name	City	Valid From	Valid Till	Entered	Superseded
John	Chennai	April 3, 1992	June 20, 2015	April 6, 1992	Jan 10, 2016
John	Mumbai	June 21, 2015	∞	Jan 10, 2016	∞

Incorporating Time in Relational Databases Using Tuple Versioning

- Add to every tuple
 - Valid start time
 - Valid end time
 - Transaction Start Time
 - Transaction End Time

Temporal Database Concepts

(a) EMP_VT

Name	<u>Ssn</u>	Salary	Dno	Supervisor_ssn	<u>Vst</u>	Vet
------	------------	--------	-----	----------------	------------	-----

DEPT_VT

Dname	<u>Dno</u>	Total_sal	Manager_ssn	<u>Vst</u>	Vet
-------	------------	-----------	-------------	------------	-----

(b) EMP_TT

Name	<u>Ssn</u>	Salary	Dno	Supervisor_ssn	<u>Tst</u>	Tet
------	------------	--------	-----	----------------	------------	-----

DEPT_TT

Dname	<u>Dno</u>	Total_sal	Manager_ssn	<u>Tst</u>	Tet
-------	------------	-----------	-------------	------------	-----

(c) EMP_BT

Name	<u>Ssn</u>	Salary	Dno	Supervisor_ssn	<u>Vst</u>	Vet	<u>Tst</u>	Tet
------	------------	--------	-----	----------------	------------	-----	------------	-----

DEPT_BT

Dname	<u>Dno</u>	Total_sal	Manager_ssn	<u>Vst</u>	Vet	<u>Tst</u>	Tet
-------	------------	-----------	-------------	------------	-----	------------	-----

Figure 24.7

Different types of temporal relational databases.
(a) Valid time database schema. (b) Transaction time database schema. (c) Bitemporal database schema.

Temporal Database Concepts

Figure 24.8

Some tuple versions in the valid time relations EMP_VT and DEPT_VT.

EMP_VT

Name	<u>Ssn</u>	Salary	Dno	Supervisor_ssn	<u>Vst</u>	Vet
Smith	123456789	25000	5	333445555	2002-06-15	2003-05-31
Smith	123456789	30000	5	333445555	2003-06-01	Now
Wong	333445555	25000	4	999887777	1999-08-20	2001-01-31
Wong	333445555	30000	5	999887777	2001-02-01	2002-03-31
Wong	333445555	40000	5	888665555	2002-04-01	Now
Brown	222447777	28000	4	999887777	2001-05-01	2002-08-10
Narayan	666884444	38000	5	333445555	2003-08-01	Now

...

DEPT_VT

Dname	<u>Dno</u>	Manager_ssn	<u>Vst</u>	Vet
Research	5	888665555	2001-09-20	2002-03-31
Research	5	333445555	2002-04-01	Now

Use of temporal database

Temporal Databases store information about states of the real world across time.

Temporal Database is a database with built-in support for handling data involving time.

It stores information relating to past, present and future time of all events.

Spatial Database Concepts

What is Spatial Data?

“Information about the location and shapes of geographic features and the relationship between them, usually stored as **coordinates** and **topology**” – ESRI

<https://www.youtube.com/watch?v=uHronVrSSdw>

Spatial Database

- ✓ **Spatial Object:** Consists of points, lines, surfaces, volumes and higher dimension objects that are used in applications of computer-aided design, cartography, geographic information system
- ✓ **Spatial Data:** The value of the objects' spatial attributes: length, configuration, perimeter, area, volume etc.
- ✓ **Spatial Database:** A spatial database is a database that is enhanced **to store and access spatial data** or data that defines a geometric space. These data are often associated with geographic locations and features, or constructed features like cities. Data on spatial databases are stored as coordinates, points, lines, polygons and topology. Some spatial databases handle more complex data like three-dimensional objects, topological coverage and linear networks.

Spatial

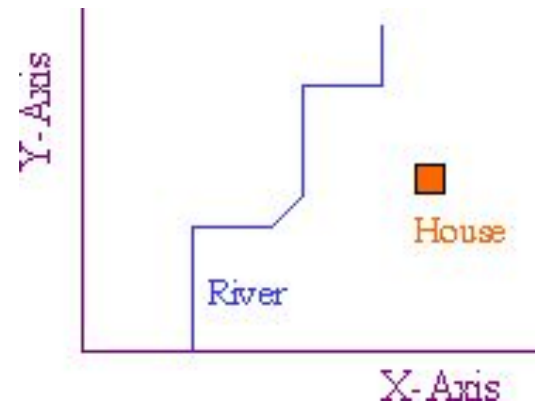
- ✓ **Spatial Database..** Spatial data, also known as geospatial data, is a term used to describe any data related to or containing information about a specific location on the Earth's surface.
- ✓ Spatial databases incorporate functionality that provides support for databases that keep track of objects in a multidimensional space. For example, cartographic databases that store maps include two-dimensional spatial descriptions of their objects-from countries and states to rivers, cities, roads, seas, and so on.
- ✓ The systems that manage geographic data and related applications are known as Geographical Information Systems (GIS), and they are used in areas such as environmental applications, transportation systems, emergency response systems, and battle management.

Spatial Representation

- ## ■ Raster model:

			R			
		R	R			
		R			H	
		R				
	R	R				
	R					
	R					

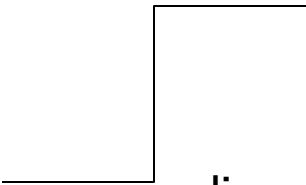
- Vector model:



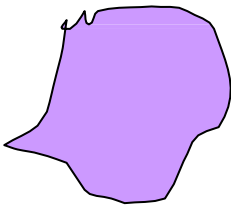
Spatial data types



point



line



region

- Point : 2 real numbers
- Line : sequence of points
- Region : area included inside n-points

Examples

- ✓ Non Spatial Query: “List the municipalities with populations over 500,000 in the Nepal”
- ✓ Spatial Query: “List crime hot spots within 10 miles of downtown Patna”

Typical types of spatial queries

- ✓ **Range query:** finds the objects of particular type that are within a given spatial area or within a particular distance from given location. For example, finds all hospitals within the Kathmandu area, or finds all ambulances within five miles of an accident location.
- ✓ **Nearest neighbor query:** Finds the objects of particular type that is closed to a given location. For example, finds the police car that is closed to a particular location.
- ✓ **Spatial joins or overlays:** Typical joins the objects of two type based on some spatial conditions, such as the objects intersecting or overlapping spatially or being within a certain distance of one another. For example, finds all cities located on a major highway.

Spatial Relationships

- Topological relationships:
 - adjacent, inside, disjoint, etc
- Direction relationships:
 - Above, below, north_of, etc
- Metric relationships:
 - “distance < 100”
- And operations to express the relationships

Spatial Queries

- Selection queries: “Find all objects inside query q ”,
inside- $\rightarrow$ intersects, north
- Nearest Neighbor-queries: “Find the closest object
to a query point q ”, k -closest objects
- Spatial join queries: Two spatial relations $S1$ and $S2$, find
all pairs: $\{x \text{ in } S1, y \text{ in } S2, \text{ and } x \text{ rel } y = \text{true}\}$, $\text{rel} = \text{intersect},$
inside, etc

Multimedia Database Concepts

Multimedia Database

- ✓ Multimedia is a combination of text, graphic, sound, animation, and video that is delivered interactively to the user by electronic or digitally manipulated means. In other words, multimedia is a form of communication that combines different content forms such as text, audio, images, animations, or video into a single interactive presentation.
- ✓ Multimedia database is the collection of interrelated multimedia data that includes text, graphics (sketches, drawings), images, animations, video, audio etc and have vast amounts of multisource multimedia data.
- ✓ These data types are broadly categorized into three classes:
 - ❖ Static media (time-independent, i.e. images and handwriting)
 - ❖ Dynamic media (time-dependent, i.e. video and sound bytes)
 - ❖ Dimensional media (i.e. 3D games or computer-aided drafting programs- CAD)

Data Types of Multimedia in Oracle

- ✓ Multimedia data types, which are represented in Oracle Database is Large Objects (LOBs).
- ✓ There are three SQL data types for defining instances of internal LOBs:
 - ❖ BLOB: A LOB whose value is composed of unstructured binary (raw) data
 - ❖ CLOB: A LOB whose value is composed of character data that corresponds to the database character set defined for the Oracle database
 - ❖ NCLOB: A LOB whose value is composed of character data that corresponds to the national character set defined for the Oracle database
 - ❖ BFILES: are large binary data objects stored in operating system files outside database table spaces

Types of multimedia data are available in current systems

- **Text:** Text is the most basic form of multimedia data. It can be stored in a variety of formats, including plain text, HTML, and XML. The text can have various types of fonts, sizes and styles. The textual data for multimedia can be developed using any text editors.
- **Images:** Images are a type of visual data that can be stored in a variety of formats, including JPEG, PNG, and GIF. There are several graphics packages available to develop graphics such as MS Paint, Corel Draw, Adobe Photoshop, Adobe Illustrator etc
- **Audio :** Audio is produced by vibration, as perceived by the sense of hearing. Audio is a type of auditory data that can be stored in a variety of formats, including WAV, MP3, and AAC.

Multimedia.....

- Video:** Video is a type of audiovisual data that can be stored in a variety of formats, including AVI, MPEG, and MP4. A set of temporally sequenced photographic data for presentation at specified rates— for example, 30 frames per second.
- Animations:** Temporal sequences of image or graphic data. A computer-based animation is a sequence of still vector image displayed in rapid succession to provide visual effect.

Challenges of Multimedia database

- ✓ Multimedia databases contains data in a large type of formats such as
.txt(text), .jpg(images), .swf(videos), .mp3(audio) etc. It is difficult to convert one type of data format to another.
- ✓ The multimedia database requires a large size as the multimedia data is quite large and needs to be stored successfully in the database.
- ✓ It takes a lot of time to process multimedia data so multimedia database is slow

Deductive Database Concepts;

Deductive Database

A deductive database can be defined as an advanced database augmented with an inference system.



By evaluating rules against facts, new facts can be derived, which in turn can be used to answer queries. It makes a database system more powerful.

What is deductive database?

- ✓ A deductive database is a database system that makes conclusions about its data based on a set of well-defined rules and facts stored in the database.
- ✓ This type of database was developed to combine logic programming with relational database management systems. Usually, the language used to define the rules and facts is the logical programming language **Datalog**.
- ✓ A deductive database is a finite collection of facts and rules. By applying the rules of a deductive database to the facts in the database, it is possible to infer additional facts, i.e. facts that are implicitly true but are not explicitly represented in the database.

Deductive

Database

A deductive database system is a database system which can make deductions (i.e.: conclude additional facts) based on rules and facts stored in the (deductive) database.

Deductive: towards the consequences:

✓ All swans are white.

✓ Tessa is a swan.

✓ **Tessa is white.**

Given the truth of the assumptions, a valid deduction guarantees the truth of the conclusion

Prolog/Datalog Notation

- **Predicate** has
 - a name
 - a fixed number of arguments
 - Convention:
 - Constants are numeric or character strings
 - Variables start with upper case letters
 - E.g., SUPERVISE(Supervisor, Supervisee)
 - States that Supervisor SUPERVISE(s) Supervisee

Prolog/Datalog Notation

■ Rule

- Is of the form head :- body
 - where :- is read as if and only iff
- E.g., SUPERIOR(X,Y) :- SUPERVISE(X,Y)
- E.g., SUBORDINATE(Y,X) :- SUPERVISE(X,Y)

Prolog/Datalog Notation

■ Query

- Involves a predicate symbol followed by y some variable arguments to answer the question
 - where :- is read as if and only iff
- E.g., SUPERIOR(james,Y)?
- E.g., SUBORDINATE(james,X)?

Figure 24.11

■ (a) Prolog notation (b) Supervisory tree

(a)

Facts

```
SUPERVISE(franksin, john).  
SUPERVISE(franksin, ramesh).  
SUPERVISE(franksin, joyce).  
SUPERVISE(jennifer, alicia).  
SUPERVISE(jennifer, ahmad).  
SUPERVISE(james, franklin).  
SUPERVISE(james, jennifer).  
...
```

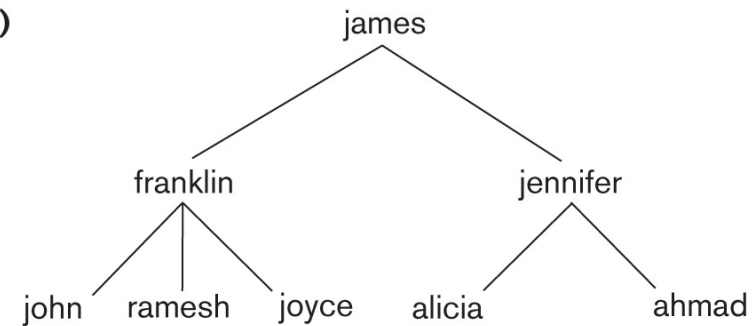
Rules

```
SUPERIOR(X, Y) :- SUPERVISE(X, Y).  
SUPERIOR(X, Y) :- SUPERVISE(X, Z), SUPERIOR(Z, Y).  
SUBORDINATE(X, Y) :- SUPERIOR(Y, X).
```

Queries

```
SUPERIOR(james, Y)?  
SUPERIOR(james, joyce)?
```

(b)



Syntax in prolog

Variables and Names

Variables begin with an uppercase letter. Predicate names, function names, and the names for objects must

begin with a lowercase letter. Rules for forming names are the same as for the predicate calculus.

Facts

A fact is a predicate expression that makes a declarative statement about the problem domain. Facts are statements that are known to be true. Whenever a variable occurs in a Prolog expression, it is assumed to be universally quantified. Note that all Prolog sentences must end with a period.

likes(john, susie). /* John likes Susie */

likes(X, susie). /* Everyone likes Susie */

likes(john, Y). /* John likes everybody */

likes(john, Y), likes(Y, john). /* John likes everybody and everybody likes John

/ likes(john, susie); likes(john, mary). / John likes Susie or John likes Mary */

not(likes(john, pizza)). /* John does not like pizza */

likes(john, susie) :- likes(john, mary). /* John likes Susie if John likes Mary.

Rule

A rule is a predicate expression that uses logical implication (:-) to describe a relationship among facts. Thus a Prolog rule takes the form

left_hand_side :- right_hand_side .

This sentence is interpreted as: **left_hand_side** if **right_hand_side**. The **left_hand_side** is restricted to a single, positive, literal, which means it must consist of a positive atomic expression. It cannot be negated and it cannot contain logical connectives.

Query:

Whenever you run the Prolog interpreter, it will **prompt** you with **?-**.

male(rati).
male(ram).
male(himanshu).
female(bina).
female(naru).
female(gunjan).
female(smriti).
parent(rati,ram).
parent(naru,ram).
parent(ram, himanshu).
parent(ram, gunjan).
parent(ram, smriti).
parent(bina, himanshu).
parent(bina, gunjan).
parent(bina, smriti).
father(X,Y):- male(X),parent(X,Y).
mother(X,Y):-female(X),parent(X,Y).
grandfather(X,Y):- male(X),parent(X,Z), parent(Z,Y).
grandmother(X,Y):- female(X),parent(X,Z),parent(Z,Y).

Lab

1. Who is the father of
gunjan?

? – father(X,gunjan)

1. Who is the grand mother
of smriti?

- grandmother(X, smriti)

[Download SWI-Prolog](#)

Datalog

Datalog is a declarative programming language that is used for querying and manipulating relational databases.

It is based on logic programming and is often used in the field of database management systems.

Declarative Language: Datalog is a declarative language, meaning you specify what you want to achieve rather than how to achieve it. You define rules and facts to represent knowledge and relationships in the database.

Rule-Based Logic: Datalog uses rules to derive new facts based on existing facts and relationships. Rules consist of a head (representing a new fact) and a body (specifying conditions for inference).

Querying the Database: Datalog allows you to retrieve information from the database using queries. Queries consist of predicates and variables that represent the desired values. The Datalog engine finds matching results based on the specified conditions.

Example

Facts are statements that are known to be true.

fact: student(john)

Rules are statements that describe how to derive new facts from existing facts.

Rule: takes(X, C) :- student(X), course(C)

The :- symbol means "if". So, the above rule states that if X is a student and C is a course, then X must be taking course C.

Queries are statements that ask for information from a Datalog program.

query: student(X) :- takes(X, C)

This query will return a list of all of the students who are taking a

Examples

Facts:

course(math)
student(john)
student(mary)

Rules:

takes(X, C) :- student(X), course(C)
teaches(X, C) :- professor(X), course(C)
student(X) :- takes(X, C)
professor(X) :- teaches(X, C)

Queries:

course(C) :- takes(X, C)
professor(X) :- teaches(X, math)
student(X) :- takes(X, math)
student(X) :- student(Y), X != Y

Introduction to Information Retrieval and Web Search

Introduction to Information Retrieval and Web Search

Information retrieval (IR) is the process of searching for and retrieving information from a collection of documents.

Web search is a specific type of IR that is used to search for information on the World Wide Web.

IR systems typically use a combination of techniques to retrieve information, including:

- **Indexing:** The documents in the collection are indexed, which means that they are assigned a unique identifier and their contents are analyzed to identify keywords and phrases.
- **Query processing:** The user's query is processed to identify the keywords and phrases that are relevant to the search.
- **Retrieval:** The system retrieves the documents that contain the keywords and phrases that are relevant to the query.

IR and Web Search

- **Web search** engines use a variety of techniques to index and retrieve web pages, including crawling, indexing, query processing, and retrieval.
- The quality of an IR system is measured by its **effectiveness and efficiency**.
- **Effectiveness** refers to the ability of the system to retrieve relevant documents.
- **Efficiency** refers to the speed with which the system can retrieve documents.
- Web search is a challenging problem because the web is a very large and ever-changing collection of documents. In addition, the documents on the